

本篇文章主要研究这个话题：**SpringFramework 如何在运行期动态注册新的数据源？**

需求来源

这个需求的起源是来自一个 SpringBoot 自动装配的数据源注册，因为一个项目中需要注册的数据源不确定，所以需要在启动时根据配置文件的内容动态注册多个数据源。后来聊着聊着，就演变成运行时动态注册新的数据源了。虽然看上去这两个事情好像差不多，但实际上两件事差了很多哈。

- 启动时动态初始化数据源：在基于 SpringFramework / SpringBoot 的应用初始化，也即 IOC 容器初始化时，读取并解析配置文件，构造多个数据源，并注册到 IOC 容器中
 - 此时通常情况下 IOC 容器还没有刷新完毕，项目还没有启动完成
- 运行期动态注册新的数据源：在项目的运行期间，动态的构造数据源，并注册到 Spring 的 IOC 容器中
 - 此时项目已经在正常运行中了

前者的处理方式相对比较简单，通过声明一个标注了 `@ConfigurationProperties` 的类，并用 Map 接收数据源的参数就可以把数据源的定义信息都获取到了：

```
1  @ConfigurationProperties(prefix = "spring.datasource.dynamic")
2  public class DynamicDataSourceProperties {
3
4      private Map<String, DataSource> dataSourceMap = new HashMap<>();
5
6      // .....
7  }
```

然后，再编写一个 `ImportBeanDefinitionRegistrar`，读取这个 `DynamicDataSourceProperties` 的内容，就可以把这些数据源都注册到 IOC 容器中。

但是后者就麻烦了，运行期动态注册新的数据源应该如何实现才行呢？下面我们来通过几个方案，讲解该需求的实现。

编码环境搭建

首先，我们先来搭建一下编码环境。

数据库准备

首先，我们先来创建 3 个不同的数据库（当然也可以只创建一个数据库，这里我们搞的更真实一点吧）：

```
1  CREATE DATABASE db1;
2  CREATE DATABASE db2;
3  CREATE DATABASE db3;
```

接下来给每一个数据库中都初始化一张相同的表：

```
1 CREATE TABLE tbl_user (
2     id int(11) NOT NULL AUTO_INCREMENT,
3     name varchar(32) NOT NULL,
4     tel varchar(16) NULL,
5     PRIMARY KEY (id)
6 );
```

OK 就这么简单的准备一下就可以了。

初始代码编写

为了快速编码，我们仍然采用 SpringBoot 构建项目，直接使用 SpringInitializer 就挺好，当然也可以通过 Maven 构建项目，这里我们就省去那些麻烦的构建步骤了，只把代码贴一下哈。

项目名称：`dynamic-register-datasource`。

`pom.xml`：

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
5         http://maven.apache.org/xsd/maven-4.0.0.xsd">
6     <modelVersion>4.0.0</modelVersion>
7     <parent>
8         <groupId>org.springframework.boot</groupId>
9         <artifactId>spring-boot-starter-parent</artifactId>
10        <version>2.2.8.RELEASE</version>
11        <relativePath/>
12    </parent>
13    <groupId>com.linkedbear.spring</groupId>
14    <artifactId>dynamic-register-datasource</artifactId>
15    <version>1.0-SNAPSHOT</version>
16
17    <properties>
18        <java.version>1.8</java.version>
19    </properties>
20
21    <dependencies>
22        <dependency>
23            <groupId>org.springframework.boot</groupId>
24            <artifactId>spring-boot-starter-web</artifactId>
25        </dependency>
26        <dependency>
27            <groupId>org.springframework.boot</groupId>
28            <artifactId>spring-boot-starter-jdbc</artifactId>
29        </dependency>
30        <dependency>
31            <groupId>mysql</groupId>
32            <artifactId>mysql-connector-java</artifactId>
33            <version>5.1.47</version>
34        </dependency>
35    </dependencies>
```

```

35
36     <build>
37         <plugins>
38             <plugin>
39                 <groupId>org.springframework.boot</groupId>
40                 <artifactId>spring-boot-maven-plugin</artifactId>
41             </plugin>
42         </plugins>
43     </build>
44 </project>

```

application.yml :

```

1  spring:
2    datasource:
3      db1:
4        driver-class-name: com.mysql.jdbc.Driver
5        jdbc-url: jdbc:mysql://localhost:3306/db1?characterEncoding=utf8 # 注意这里是jdbc-url而不
    是url
6        username: root
7        password: 123456
8      db2:
9        driver-class-name: com.mysql.jdbc.Driver
10       jdbc-url: jdbc:mysql://localhost:3306/db2?characterEncoding=utf8
11       username: root
12       password: 123456

```

DataSourceConfiguration :

```

1  @Configuration
2  public class DataSourceConfiguration {
3
4      @Bean
5      @Primary
6      @ConfigurationProperties("spring.datasource.db1")
7      public DataSource db1() {
8          return DataSourceBuilder.create().build();
9      }
10
11     @Bean
12     @ConfigurationProperties("spring.datasource.db2")
13     public DataSource db2() {
14         return DataSourceBuilder.create().build();
15     }
16 }

```

以上的代码，是我们最常见到的 SpringBoot 中定义多个数据源的方法了是吧。

测试运行一下

最后编写 SpringBoot 主启动类，在这里我们将启动完成后的 IOC 容器拿到，并从中取出所有的 `DataSource`，取一下它们其中的数据库连接 `Connection`：

```
1  @SpringBootApplication
2  public class DynamicRegisterDataSourceApplication {
3
4      public static void main(String[] args) throws Exception {
5          ConfigurableApplicationContext ctx =
SpringApplication.run(DynamicRegisterDataSourceApplication.class, args);
6          Map<String, DataSource> dataSourceMap = ctx.getBeansOfType(DataSource.class);
7          for (Map.Entry<String, DataSource> entry : dataSourceMap.entrySet()) {
8              String name = entry.getKey();
9              DataSource dataSource = entry.getValue();
10             System.out.println(name);
11             System.out.println(dataSource.getConnection()); // 这里会抛出异常，直接throws走了
12         }
13     }
14 }
```

运行主启动类，可以在控制台中发现我们已经注册好的两个 `DataSource`，以及它们对应的 `Connection`：

```
1  db1
2  2021-01-15 20:43:14.299 INFO 7624 --- [          main] com.zaxxer.hikari.HikariDataSource
   : HikariPool-1 - Starting...
3  2021-01-15 20:43:14.412 INFO 7624 --- [          main] com.zaxxer.hikari.HikariDataSource
   : HikariPool-1 - Start completed.
4  HikariProxyConnection@65982709 wrapping com.mysql.jdbc.JDBC4Connection@64030b91
5  db2
6  2021-01-15 20:43:14.414 INFO 7624 --- [          main] com.zaxxer.hikari.HikariDataSource
   : HikariPool-2 - Starting...
7  2021-01-15 20:43:14.418 INFO 7624 --- [          main] com.zaxxer.hikari.HikariDataSource
   : HikariPool-2 - Start completed.
8  HikariProxyConnection@652007616 wrapping com.mysql.jdbc.JDBC4Connection@66e889df
```

到这里，基本的环境和代码都就准备好了。

下面，我们来讲解两种程序运行期动态注册数据源的解决方案。

解决方案1：基于BeanDefinition

如果各位小伙伴有学习过我 Spring 小册的 IOC 高级部分，应该都知道 **bean 的创建来源是 `BeanDefinition`** 吧！通常情况下，我们通过 `<bean>` 标签、`@Bean` 注解，或者 `@Component` 配合 `@ComponentScan` 注解完成的 bean 注册，都是先封装为一个个的 `BeanDefinition`，然后才是根据 `BeanDefinition` 创建 bean 对象！

使用 SpringFramework 的 `BeanDefinition` 元编程，我们可以手动构造一个 `BeanDefinition`，并注册到 `DefaultListableBeanFactory`（`BeanDefinitionRegistry`）中：

```
1  @RestController
```

```

2 public class RegisterDataSourceController implements BeanFactoryAware, ApplicationContextAware
3 {
4     private DefaultListableBeanFactory beanFactory;
5
6     @GetMapping("/register1")
7     public String register1() {
8         BeanDefinitionBuilder builder =
9         BeanDefinitionBuilder.rootBeanDefinition(HikariDataSource.class);
10        builder.addPropertyReference("driverClassName", "com.mysql.jdbc.Driver");
11        builder.addPropertyReference("jdbcUrl", "jdbc:mysql://localhost:3306/db3?
12        characterEncoding=utf8");
13        builder.addPropertyReference("username", "root");
14        builder.addPropertyReference("password", "123456");
15        // builder.setScope(ConfigurableListableBeanFactory.SCOPE_SINGLETON);
16        beanFactory.registerBeanDefinition("db3", builder.getBeanDefinition());
17        return "success";
18    }
19
20    @GetMapping("/getDataSources")
21    public String getDataSources() {
22        Map<String, DataSource> dataSourceMap = beanFactory.getBeansOfType(DataSource.class);
23        dataSourceMap.forEach((s, dataSource) -> {
24            System.out.println(s + " ===== " + dataSource);
25        });
26        return "success";
27    }
28    // .....
29 }

```

如果构造的 `DataSource` 需要指定作用域等额外的配置，可以操纵 `BeanDefinitionBuilder` 的 API 进行设置。

以此法编写好之后，我们可以重启项目测试一下。重启之后先访问 `/getDataSources`，可以发现控制台只有两个 `DataSource` 的打印：

```

1 db1 ===== HikariDataSource (HikariPool-1)
2 db2 ===== HikariDataSource (HikariPool-2)

```

然后访问 `/register1` 路径，之后再访问 `/getDataSources`，控制台就可以打印三个 `DataSource` 了：

```

1 db1 ===== HikariDataSource (HikariPool-1)
2 db2 ===== HikariDataSource (HikariPool-2)
3 db3 ===== HikariDataSource (HikariPool-3)

```

这种方法比较简单，比较具有通用性，关键的点是抓住核心知识点：`BeanFactory` 中的 `bean` 绝大多数都是通过 `BeanDefinition` 创建而来。

解决方案2：基于SingletonBeanRegistry

如果需要注册的 `bean` 都是**单实例 `bean`**，而且**不需要经过 AOP 处理**的话，则也可以使用接下来要讲的这种方式，相较于上一种而言，采用这种方法相对会更友好。

如果小伙伴有看过我的 [Spring 小册第 14 章 BeanFactory](#) 章节，应该不会忘记 `BeanFactory` 在 `ApplicationContext` 中唯一现役的最终实现是 `DefaultListableBeanFactory` 吧。那这个实现类，最终是继承了 `AbstractBeanFactory`，而它又继承了一个叫 `DefaultSingletonBeanRegistry` 的类，这个类我们在 Spring 小册的正篇中没有提及，现已经补充到[小册的加餐内容](#)中了，小伙伴们可以戳链接去学习呀。

简单示例代码

我们简单的来说哈，`DefaultSingletonBeanRegistry` 这个类实现了一个 `SingletonBeanRegistry` 接口，这个接口中定义了一个方法：`registerSingleton`，它可以直接向 IOC 容器注册一个已经完完全全存在的对象，使其成为 IOC 容器中的一个 bean。

```
1 void registerSingleton(String beanName, Object singletonObject);
```

又因为 `DefaultListableBeanFactory` 继承自 `DefaultSingletonBeanRegistry`，所以借助这个原理之后，实现这个需求就简单的很了。我们只需要拿到 `DefaultListableBeanFactory`，之后调用它的 `registerSingleton` 方法即可：

```
1 @RestController
2 public class RegisterDataSourceController implements BeanFactoryAware {
3
4     private DefaultListableBeanFactory beanFactory;
5
6     @GetMapping("/getDataSources")
7     public String getDataSources() {
8         Map<String, DataSource> dataSourceMap = beanFactory.getBeansOfType(DataSource.class);
9         dataSourceMap.forEach((s, dataSource) -> {
10             System.out.println(s + " ===== " + dataSource);
11         });
12         return "success";
13     }
14
15     @GetMapping("/register2")
16     public String register2() throws SQLException {
17         HikariDataSource dataSource = new HikariDataSource();
18         dataSource.setDriverClassName("com.mysql.jdbc.Driver");
19         dataSource.setJdbcUrl("jdbc:mysql://localhost:3306/db3?characterEncoding=utf8");
20         dataSource.setUsername("root");
21         dataSource.setPassword("123456");
22         dataSource.getConnection();
23         System.out.println("db3 创建完成!");
24         beanFactory.registerSingleton("db3", dataSource);
25         return "success";
26     }
27
28     @Override
29     public void setBeanFactory(BeanFactory beanFactory) throws BeansException {
30         this.beanFactory = (DefaultListableBeanFactory) beanFactory;
31     }
32 }
```

这样注册好了，IOC 容器中就有这个 db3 的数据源了，我们可以再测试一下。

测试注册效果

重启工程，先访问 `/getDataSources`，控制台依然是只有两个 `DataSource` 的打印：

```
1 db1 ===== HikariDataSource (HikariPool-1)
2 db2 ===== HikariDataSource (HikariPool-2)
```

然后访问 `/register2` 路径，控制台可以打印成功 `db3 创建完成!`，此时再访问 `/getDataSources` 路径，控制台也可以打印三个 `DataSource` 了：

```
1 db1 ===== HikariDataSource (HikariPool-1)
2 db2 ===== HikariDataSource (HikariPool-2)
3 db3 ===== HikariDataSource (HikariPool-3)
```

依赖注入？

上面我们看到的生效，那仅仅是我们拿到 `BeanFactory`，或者 `ApplicationContext` 后主动调用 `getBean` 系列方法，去获取 IOC 容器的 bean。但对于那些依赖了 `DataSource` 的 bean，这种情况就不好办了：**因为依赖注入的时机是 bean 的初始化阶段，当 bean 创建完成后，没有其他代码的干涉，bean 依赖的那些 bean 就不会变化。**

听起来有点绕，我们来写一个 Service 类来解释一下。

```
1 @Service
2 public class DataSourceService {
3
4     @Autowired
5     Map<String, DataSource> dataSourceMap;
6
7     public void printDataSources() {
8         dataSourceMap.forEach((s, dataSource) -> {
9             System.out.println(s + " ===== " + dataSource);
10        });
11    }
12 }
```

这里我们造了一个 `DataSourceService`，并通过注入一整个 `Map` 的方式，将 IOC 容器中的 `DataSource` 连带着 bean 的 name 都注入进来。

然后我们修改一下 Controller，让它取容器中的 `DataSourceService`，打印它里面的 `DataSource`：

```
1 @GetMapping("/getDataSources")
2 public String getDataSources() {
3     DataSourceService dataSourceService = beanFactory.getBean(DataSourceService.class);
4     dataSourceService.printDataSources();
5     return "success";
6 }
```

重启工程，并重复上面的测试效果，这次发现两次打印的结果是一样的：

```
1 db1 ===== HikariDataSource (HikariPool-1)
2 db2 ===== HikariDataSource (HikariPool-2)
3 db3 创建完成!
4 db1 ===== HikariDataSource (HikariPool-1)
5 db2 ===== HikariDataSource (HikariPool-2)
```

这个现象就是上面提到的：**bean 中依赖注入的属性没有被主动干预，则不会发生变化。**

怎么解决这个问题呢？哎，还是靠 `BeanFactory` 。

在 [Spring 小册第 14 章 BeanFactory](#) 的 1.4.2 节中我们讲到了有关 `AutowireCapableBeanFactory` 的一个作用是**框架集成**，它提供了一个 `autowireBean` 方法，用于给现有的对象进行依赖注入：

```
1 void autowireBean(Object existingBean) throws BeansException;
```

所以我们可以借助这个特性，在动态注册完 `DataSource` 后，把 IOC 容器中的 `DataSourceService` 取出来，让它重新执行一次依赖注入即可：

```
1 @GetMapping("/register2")
2 public String register2() throws SQLException {
3     HikariDataSource dataSource = new HikariDataSource();
4     dataSource.setDriverClassName("com.mysql.jdbc.Driver");
5     dataSource.setJdbcUrl("jdbc:mysql://localhost:3306/db3?characterEncoding=utf8");
6     dataSource.setUsername("root");
7     dataSource.setPassword("123456");
8     dataSource.getConnection();
9     System.out.println("db3 创建完成!");
10    beanFactory.registerSingleton("db3", dataSource);
11    // 重新执行依赖注入
12    beanFactory.autowireBean(beanFactory.getBean(DataSourceService.class));
13    return "success";
14 }
```

就这么简单，添加这样一行代码即可。

再测试

OK，重新测试一下效果怎样，重启工程，按照上面的测试过程，先访问 `/getDataSources`，再访问 `/register2`，然后重新访问 `/getDataSources`，这次控制台打印了 `DataSourceService` 中的 3 个 `DataSource` 了：

```
1 db1 ===== HikariDataSource (HikariPool-1)
2 db2 ===== HikariDataSource (HikariPool-2)
3 db3 创建完成!
4 db1 ===== HikariDataSource (HikariPool-1)
5 db2 ===== HikariDataSource (HikariPool-2)
6 db3 ===== HikariDataSource (HikariPool-3)
```

这样依赖注入的问题也就解决了。

不足？

虽然上面这样的写法没啥问题，但如果依赖 `DataSource` 的 bean 太多，那我们一个一个的重新依赖注入，那岂不是太费劲了？有没有更好的方案，能**针对某一种特定的 bean 的类型，当 `BeanFactory` 动态注册该类型的 bean 时，自动刷新 IOC 容器中依赖了该类型 bean 的 bean**。这个想法是否能实现呢？

优化方案：自定义注解+事件监听

比较可惜，使用普通的套路我们无法比较容易的获取到 IOC 容器中哪些 bean 依赖这些 `DataSource`，所以我们可以换一个思路：既然依赖这些 `DataSource` 的 bean 通常都是我们自己编写的（我们自己的业务场景需要呀），所以我们可以完全给这些 bean 上面添加一个自定义的注解。

自定义注解

譬如说，我们给上面的代码中，`DataSourceService` 的上面添加一个 `@RefreshDependency` 注解：

```
1  @Documented
2  @Retention(RetentionPolicy.RUNTIME)
3  @Target(ElementType.TYPE)
4  public @interface RefreshDependency {
5
6  }
```

```
1  @Service
2  @RefreshDependency
3  public class DataSourceService {
4
5      @Autowired
6      Map<String, DataSource> dataSourceMap;
7
8      // .....
9  }
```

这个注解的作用，就是标识那些需要 `BeanFactory` 去执行依赖重注入动作的 bean。

接下来，就是每次动态注册完 bean 后，让 `BeanFactory` 去寻找这些标有 `@RefreshDependency` 注解的 bean，并执行依赖重注入：

```
1  @GetMapping("/register3")
2  public String register3() throws SQLException {
3      HikariDataSource dataSource = new HikariDataSource();
4      dataSource.setDriverClassName("com.mysql.jdbc.Driver");
5      dataSource.setJdbcUrl("jdbc:mysql://localhost:3306/db3?characterEncoding=utf8");
6      dataSource.setUsername("root");
7      dataSource.setPassword("123456");
8      dataSource.getConnection();
9      System.out.println("db3 创建完成!");
10     beanFactory.registerSingleton("db3", dataSource);
11
12     Map<String, Object> beansMap =
13     beanFactory.getBeansWithAnnotation(RefreshDependency.class);
14     beansMap.values().forEach(bean -> beanFactory.autowireBean(bean));
```

```
14         return "success";
15     }
```

当然，这两行代码虽然不长，但它毕竟是一个可以抽取的逻辑。如果后续我们的代码中还有别的地方也需要动态注册新的 bean 后通知其它 bean 完成依赖重注入，则相同的代码又要再写一次。

针对这个问题，我们可以继续使用事件驱动的特性来优化。

事件驱动优化

既然要用事件驱动，而我们又知道 `ApplicationContext` 本身也是一个 `ApplicationEventPublisher`，它具备发布事件的能力，所以我们这次就不必在 Controller 中注入 `BeanFactory` 了，而是换用 `ApplicationContext`：

```
1  @RestController
2  public class RegisterDataSourceController implements BeanFactoryAware, ApplicationContextAware
3  {
4      private ConfigurableApplicationContext ctx;
5
6      // .....
7
8      @Override
9      public void setApplicationContext(ApplicationContext ctx) throws BeansException {
10         this.ctx = (ConfigurableApplicationContext) ctx;
11     }
12 }
```

注意这里要用 `ConfigurableApplicationContext` 去接收，因为 `ApplicationContext` 接口并没有继承 `SingletonBeanRegistry` 接口，`ConfigurableApplicationContext` 才继承了它。

然后，在注册完 bean 之后，就可以发布一个事件，通过事件机制来触发 bean 的依赖重注入了。我们先来把事件和监听器造出来：

```
1  // 继承自ApplicationContextEvent，则可以直接从事件中获取ApplicationContext
2  public class DynamicRegisterEvent extends ApplicationContextEvent {
3
4      public DynamicRegisterEvent(ApplicationContext source) {
5          super(source);
6      }
7  }
```

```

1  @Component
2  public class DynamicRegisterListener implements ApplicationListener<DynamicRegisterEvent> {
3
4      @Override
5      public void onApplicationEvent(DynamicRegisterEvent event) {
6          ApplicationContext ctx = event.getApplicationContext();
7          AutowireCapableBeanFactory beanFactory = ctx.getAutowireCapableBeanFactory();
8          Map<String, Object> beansMap = ctx.getBeansWithAnnotation(RefreshDependency.class);
9          beansMap.values().forEach(beanFactory::autowireBean);
10     }
11 }

```

OK，把监听器注册到 IOC 容器周，接下来再修改 Controller 中的动态注册 bean 的逻辑，让它注册完 bean 后发布 `DynamicRegisterEvent` 事件：

```

1  @GetMapping("/register3")
2  public String register3() throws SQLException {
3      HikariDataSource dataSource = new HikariDataSource();
4      dataSource.setDriverClassName("com.mysql.jdbc.Driver");
5      dataSource.setJdbcUrl("jdbc:mysql://localhost:3306/db3?characterEncoding=utf8");
6      dataSource.setUsername("root");
7      dataSource.setPassword("123456");
8      dataSource.getConnection();
9      System.out.println("db3 创建完成!");
10     ctx.getBeanFactory().registerSingleton("db3", dataSource);
11
12     ctx.publishEvent(new DynamicRegisterEvent(ctx));
13     return "success";
14 }

```

这样一切就大功告成了，注册 bean 的逻辑，和依赖重注入的逻辑也都通过事件驱动解耦了。

重新测试一下，浏览器先后访问 `/getDataSources`、`/register3`、`/getDataSources`，控制台依然可以打印 `DataSourceService` 中的 3 个 `DataSource`：

```

1  db1 ===== HikariDataSource (HikariPool-1)
2  db2 ===== HikariDataSource (HikariPool-2)
3  db3 创建完成!
4  db1 ===== HikariDataSource (HikariPool-1)
5  db2 ===== HikariDataSource (HikariPool-2)
6  db3 ===== HikariDataSource (HikariPool-3)

```

说明我们的优化方案是没有问题的。

本文涉及到的所有源码可以从 GitHub 中找到：<https://github.com/LinkedBear/juejin-posts-sources>

【都看到这里了，小伙伴们要不要关注点赞一下呀，有 Spring、SpringBoot、MyBatis 及相关源码学习需要的可以看我的柯基小册四件套哦（对，是四件套了），学习起来～奥利给】